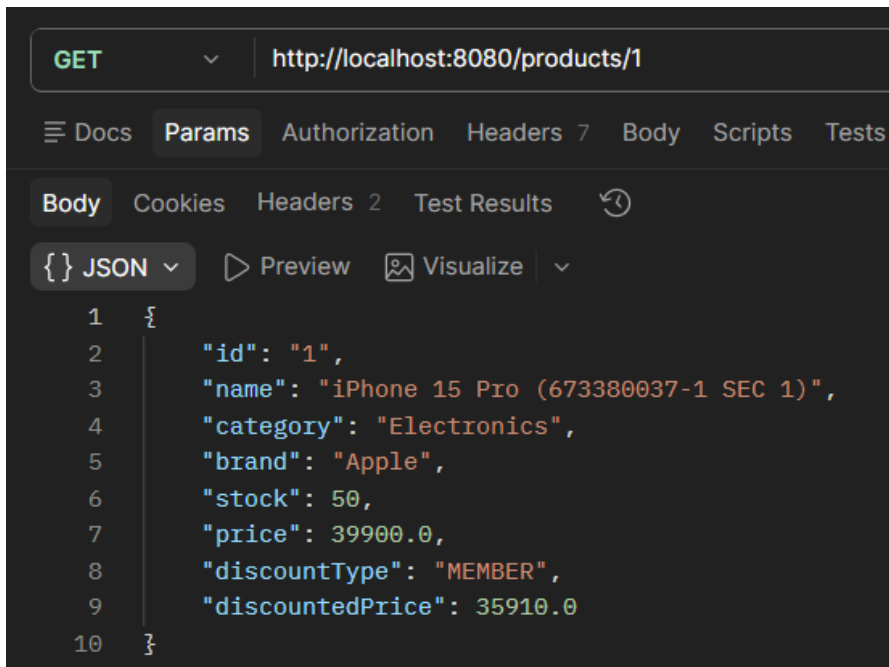


ภาพ Screenshot ทดสอบทุก endpoint ผ่าน Postman

GET : /products/{id}



Mono<Product> คือ object ที่ id = 1 คือ iPhone 15 Pro พร้อม discountedPrice: 35910.0 ซึ่งคำนวณจาก $39900 * 0.90$ เพราะ discountType = "MEMBER" ลด 10%

```

public Mono<Product> findById(String id) {
    // TODO: เติม code ตรงนี้
    Product product = store.get(id);
    return product != null ? Mono.just(product) : Mono.empty();
}

```

- store.get(id) ดึงข้อมูลจาก ConcurrentHashMap ในหน่วยความจำแบบ Synchronous ตรงๆ ไม่ต้องรอ I/O
- Reactive Streams ห้ามส่งค่า null หากใช้ Mono.just(null) จะเกิด NullPointerException ทันที
- ต้องเช็ค null เพื่อแยกส่งค่า หากพบข้อมูลให้คืน Mono.just(product) หากไม่พบให้คืน Mono.empty()
- Mono.empty() หมายถึงไม่มีข้อมูล ซึ่งชั้น Service จะนำไปแปลงเป็น error ผ่าน switchIfEmpty ในภายหลัง

GET : /products



```
GET http://localhost:8080/products

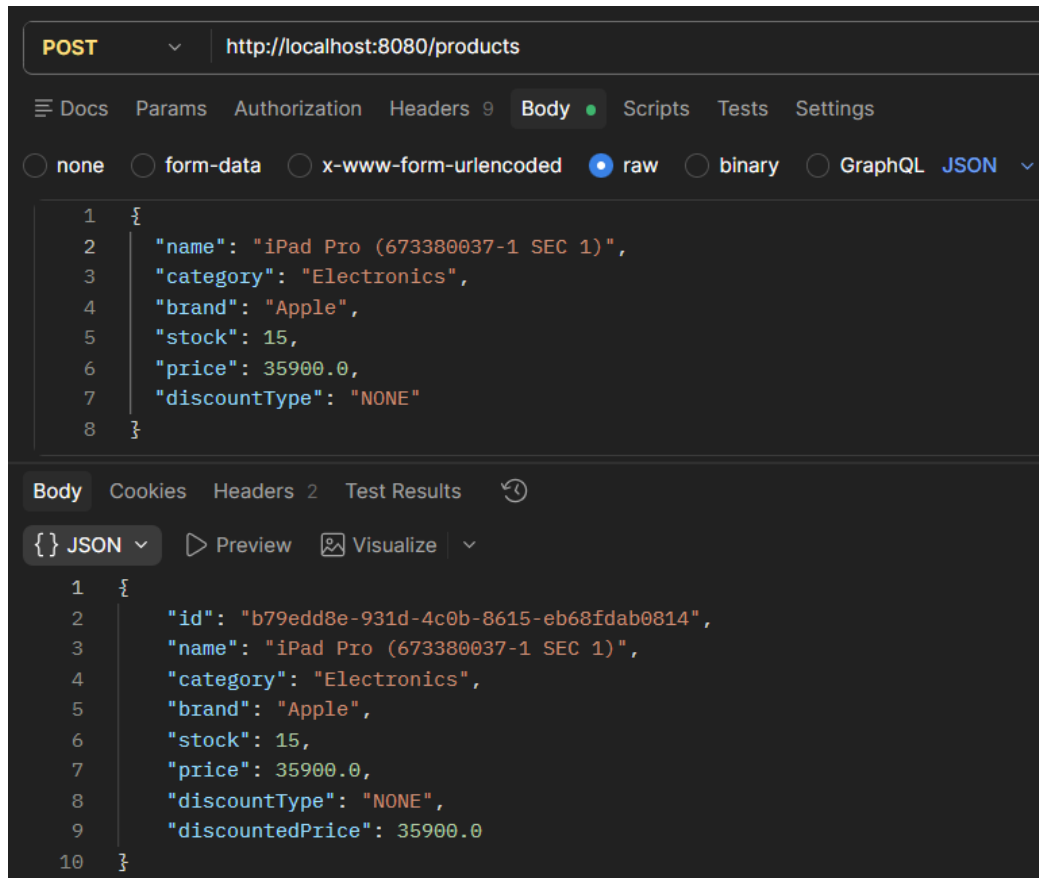
Body
JSON
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380037-1 SEC 1)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900.0,
    "discountType": "NONE",
    "discountedPrice": 49900.0
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900.0,
    "discountType": "SEASONAL",
    "discountedPrice": 23920.0
  }
]
```

Flux<Product> คื array 3 รายการ (id 1, 2, 3) แสดงว่า findAll() แปลง store.values() เป็น Flux ได้สำเร็จ และแต่ละตัวคำนวณส่วนลดถูกต้องตาม discountType ของตัวเอง MEMBER = ลด10%, NONE = ไม่ลด, SEASONAL = ลด 20% คำนวณ $29900 \times 0.8 = 23920.0$ ตรงกับผลลัพธ์

```
public Flux<Product> findAll() {
    // TODO: เติม code ตรงนี้
    return Flux.fromIterable(store.values()); // ← แก้นรทัดนี้
}
```

- store.values() คืค่าเป็น Collection<Product> ของข้อมูลทั้งหมดที่มีอยู่ในหน่วยความจำขณะนั้น
- Flux.fromIterable(...) ทำหน้าที่แปลง Collection ให้เป็น Reactive Stream เพื่อให้ชั้น Controller นำไปต่อเชื่อม Operator เช่น .filter() หรือ .map() ต่อได้ทันทีโดยไม่มีคความหน่วง

POST : /products

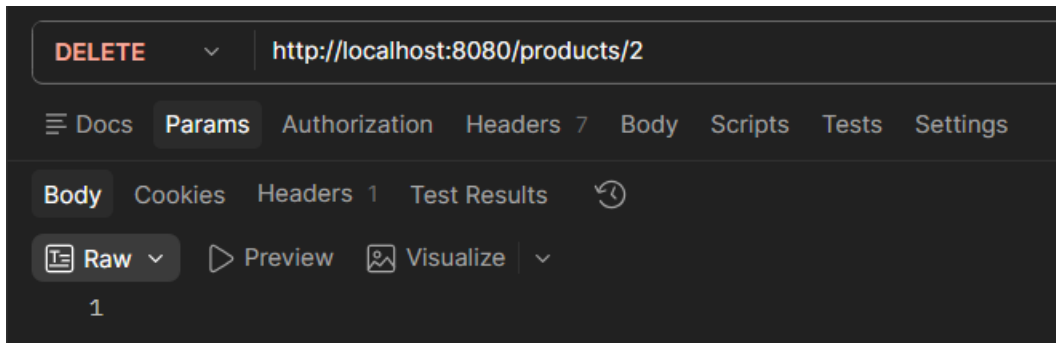


Mono<Product> ส่ง body ที่ไม่มี id เข้าไป ระบบ generate UUID ให้อัตโนมัติ (b79edd8e-...) แสดงว่า logic ใน ProductService.save() ที่ใช้ if (product.getId() == null) ทำงานถูกต้อง และ discountedPrice คำนวณให้ทันที ตอนคืนค่า

```
public Mono<Product> save(Product product) {
    // TODO: เติม code ตรงนี้
    store.put(product.getId(), product);
    return Mono.just(product);
}
```

- store.put(...) เป็นคำสั่งที่มี Side Effect คือส่งแก้ไขข้อมูลใน Map ทันทีที่เรียกฟังก์ชัน
- store.put(...) ทำงานทันทีแบบ Eager ไม่เหมือน Operator ทั่วไปของ Reactor ที่เป็น Lazy (รอการ Subscribe ก่อน)
- Mono.just(product) คืนค่าวัตถุเดิมกลับไปเพื่อยืนยันว่าการบันทึกสำเร็จพร้อมข้อมูลล่าสุด

DELETE : /products/{id}

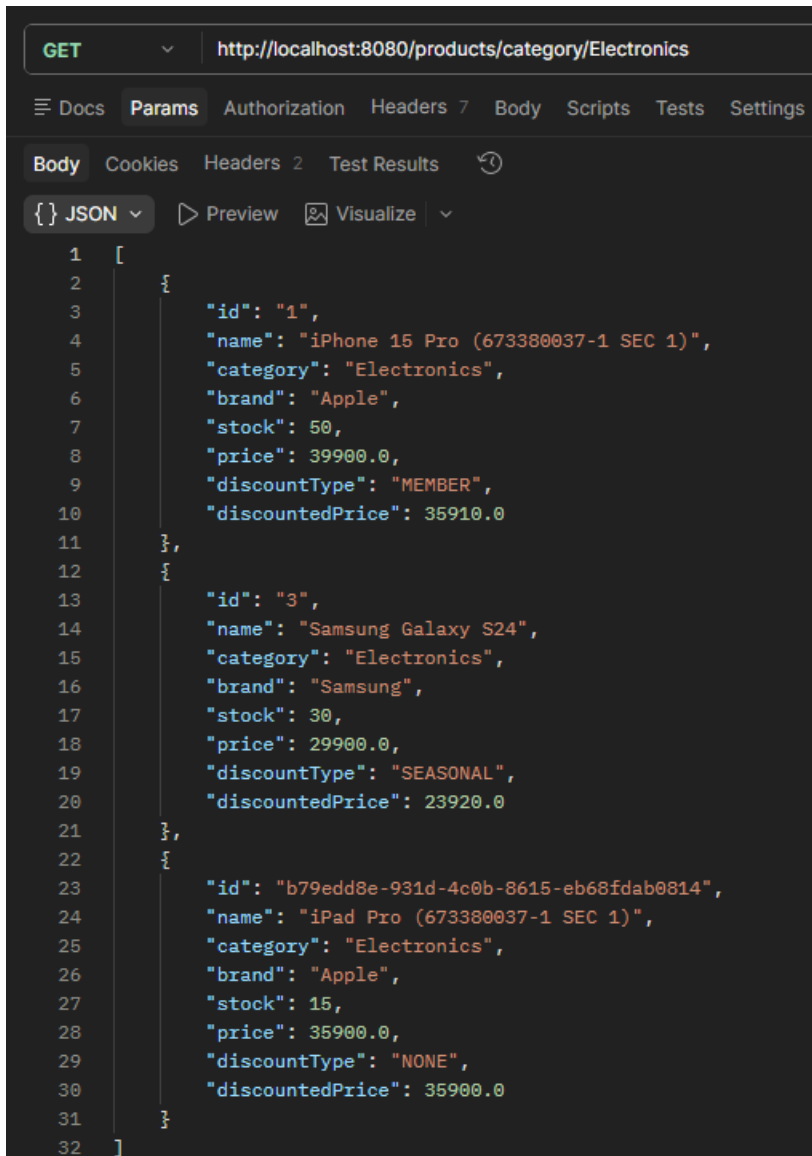


response ว่างเปล่า เพราะ endpoint คืน Mono<Void> ไม่มี body ให้แสดง นี่คือพฤติกรรมปกติของ reactive delete ที่สำเร็จ

```
public Mono<Void> deleteById(String id) {  
    // TODO: เติม code ตรงนี้  
    store.remove(id);  
    return Mono.empty(); // ← แก่บรรทัดนี้  
}
```

- Mono<Void> คือประเภทข้อมูลที่แจ้งว่าทำงานสำเร็จแต่ไม่มี Payload คืนกลับมา
- Mono.empty() ในบริบทการลบข้อมูลหมายถึงการทำงานเสร็จสิ้นสมบูรณ์ (Complete) ไม่ได้แปลว่าหาไม่พบหรือเกิดข้อผิดพลาด
- Postman จะแสดง Response ว่างเปล่าเมื่อทดสอบคำสั่ง DELETE เนื่องจากไม่มีข้อมูลส่งกลับ

GET : /products/category/{cat}

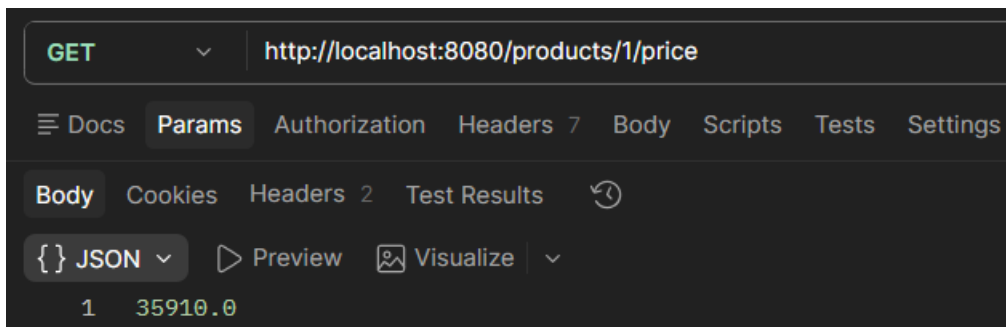


Flux<Product> ค็นสินค้าที่เหลือหลังลบ id = 2 ออกไปแล้ว (เหลือ id 1, 3, และ iPad Pro ที่เพิ่งเพิ่ม) ทั้งหมดมี category = "Electronics" ตรงตาม filter findByCategory() ทำงานถูกต้อง

```
public Flux<Product> findByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return findAll().filter(product -> product.getCategory().equalsIgnoreCase(category));
}
```

- ฟังก์ชันนี้ใช้การต่อยอด (Compose) Stream ด้วยการเรียกใช้ findAll() ซ้ำ แทนการเขียนโค้ดดึงข้อมูลใหม่
- .filter(...) เป็น Operator แบบ Lazy จะยังไม่ทำงานจนกว่าจะมีการ Subscribe จาก Controller
- equalsIgnoreCase ช่วยให้ค้นหาเจอโดยไม่สนใจตัวพิมพ์เล็กหรือตัวพิมพ์ใหญ่

GET : /products/{id}/price



Mono<Double> คืตัวเลข 35910.0 เพราะ endpoint นี้คืน Mono<Double> โดยตรง ไม่ได้ห่อด้วย object ใด ๆ

```
public Mono<Double> getDiscountedPrice(String id) {  
    // TODO: เติม code ตรงนี้  
    return getById(id)  
        .map(Product::getDiscountedPrice);  
}
```

- เรียกใช้ getById(id) ในคลาสเดิมเพื่อใช้ประโยชน์จาก switchIfEmpty หากไม่พบ ID จะตัดจบเป็น error ทันทีโดยไม่เข้า .map()
- .map(Product::getDiscountedPrice) เป็นการใช้ Method Reference ซึ่งทำงานเหมือนกับ .map(p -> p.getDiscountedPrice()) ทุกประการ
- .map() ใช้แปลงประเภท Stream จาก Mono<Product> เป็น Mono<Double> แบบ Synchronous เพราะเป็นการคำนวณในหน่วยความจำ หากต้องต่อ I/O แบบ Asynchronous ต้องเปลี่ยนไปใช้ .flatMap()

Reactive Programming vs Blocking — ต่างกันอย่างไร

การทำงานฝั่ง Server แบ่งเป็น 2 รูปแบบหลัก

- Blocking (แบบดั้งเดิม): ใช้โมเดล 1 Request = 1 Thread ผ่าน Thread Pool เช่น Tomcat เมื่อสั่งดึงข้อมูลอย่าง `Product p = repository.findById(id);` ตัว Thread จะหยุดยืนรอ I/O จนกว่าจะได้ผลลัพธ์ ทำให้สิ้นเปลืองทรัพยากรสูงเมื่อมี Request เข้ามาพร้อมกันจำนวนมาก เหมาะกับงาน CPU-bound หรือระบบที่ Traffic ไม่สูง
- Reactive (WebFlux): รันบน Reactor Netty ใช้ Thread จำนวนน้อย (Event Loop) แบบ Non-blocking คำสั่งอย่าง `Mono<Product> p = repository.findById(id);` จะคืนค่าทันทีในลักษณะเป็น สัญญาว่าจะส่งข้อมูลมาทีหลัง โดย Thread ไม่ต้องนั่งรอ แต่ไปรับ Request อื่นต่อได้ทันที จึงประหยัดทรัพยากรและรองรับ High Traffic หรือ I/O-bound ได้ดีกว่า แม้โค้ดจะซับซ้อนกว่าก็ตาม

ใน Lab นี้ Method ทั้งหมดจึงต้องคืนค่าเป็น Mono หรือ Flux เสมอ แทนการคืนค่า Object ตรงๆ เพื่อรักษาความเป็น Reactive Stream

Mono vs Flux — ใช้กรณีไหน และทำไม

- `Mono<T>` แทนสัญญาการส่งข้อมูล 0 หรือ 1 ค่า เหมาะกับฟังก์ชันที่ต้องการผลลัพธ์เดียว เช่น `findById()`, `save()`, `getDiscountedPrice()` หรือ `deleteById()` ซึ่งคืน 0 ค่า หรือ Void
- `Flux<T>` แทนสัญญาการส่งข้อมูล 0 ถึง N ค่า ทำหน้าที่เหมือน stream หรือ list แบบ async เหมาะกับงานที่อาจได้ผลลัพธ์หลายรายการ เช่น `findAll()` หรือ `findByCategory()`

ที่ต้องแยกสองประเภท เพราะ การแยก Type ชัดเจนช่วยให้ใช้ Operator ที่เหมาะสมกับจำนวนข้อมูล เช่น ใช้ `switchIfEmpty()` กับ Mono เพื่อสื่อว่า ถ้าไม่เจอข้อมูลนี้ให้แจ้ง error หรือใช้ `.filter()` กับ Flux เพื่อคัดกรองข้อมูลหลายรายการ นอกจากนี้ยังช่วยให้ Compiler และผู้รับช่วงดูแลโค้ดเข้าใจได้ทันทีว่า endpoint นั้นส่งคืนข้อมูลเดียวหรือชุดข้อมูล โดยไม่ต้องไล่อ่าน logic ข้างใน

WebClient — อธิบาย method chain `.get()`, `.uri()`, `.retrieve()`, `.bodyToMono/Flux()`

```
public Flux<Product> getAllProducts() {  
    // TODO: เติม code ตรงนี้  
    return client.get()  
        .uri("/products")  
        .retrieve()  
        .bodyToFlux(Product.class);  
}
```

- `.get()` , `.post()` , `.delete()` — เลือก HTTP verb ที่จะส่งไปหา server ปลายทาง

- `.uri(...)` — รับ URL แบบ template string ได้ เช่น `{id}` แล้วแทนค่าพารามิเตอร์ให้อัตโนมัติ ปลอดภัยกว่าการต่อ string เอง
- `.retrieve()` — จุดนี้คือจุดที่ WebClient เริ่มส่ง request ออกไปจริง ๆ แบบ non-blocking แล้วรอรับ response แบบ async
- `.bodyToMono(T.class)` — ใช้เมื่อคาดว่า response จะเป็น object เดียว เช่น 1 product
- `.bodyToFlux(T.class)` — ใช้เมื่อคาดว่า response จะเป็น array/list เช่น product หลายตัว

ทุก method ของ WebClient ที่เขียนไปคืน Mono/Flux กลับมาทันที ยังไม่มีการส่ง request จริงจนกว่าจะมีคน `.subscribe()` ในที่นี้คือ Spring WebFlux subscribe ให้อัตโนมัติเมื่อ controller คืนค่ากลับไป นี่คือการ lazy ของ Reactive Streams

Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository

```
/**
 * ProductRepository – In-memory Reactive Repository
 *
 * ✔ โครงสร้างและ annotation ครบแล้ว
 * ✗ TODO: เติม method body ให้ครบทุก method
 *
 * ใช้ ConcurrentHashMap เป็น in-memory storage
 * (ไม่ต่อ Database – เน้นฝึก Mono/Flux)
 *
 * Hint:
 * - Mono.just(value)      คืนค่าเดียว
 * - Mono.empty()          คืนเปล่า
 * - Flux.fromIterable(list) คืนหลายค่าจาก collection
 */
public class ProductRepository {

    // — In-memory storage —————
    // ConcurrentHashMap ปลอดภัยกับ multi-thread โดยไม่ต้อง synchronized เอง
    // สำคัญในบริบท reactive ที่หลาย request อาจถูกประมวลผลพร้อมกันบน event loop
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // — Constructor: ใส่ข้อมูลตัวอย่าง —————
    public ProductRepository() {
        store.put(key: "1", new Product(id: "1", name: "iPhone 15 Pro (673380037-1 SEC 1)",
            category: "Electronics", brand: "Apple", stock: 50, price: 39900.0, discountType: "MEMBER"));
        store.put(key: "2", new Product(id: "2", name: "MacBook Air M3",
            category: "Electronics", brand: "Apple", stock: 20, price: 49900.0, discountType: "NONE"));
        store.put(key: "3", new Product(id: "3", name: "Samsung Galaxy S24",
            category: "Electronics", brand: "Samsung", stock: 30, price: 29900.0, discountType: "SEASONAL"));
    }
}
```

```

public Mono<Product> findById(String id) {
    // store.get(id) เป็นการอ่านค่าแบบ synchronous ธรรมดา (ไม่มี I/O จริง
    // เพราะเป็น in-memory map) จึงไม่ต้อง wrap เป็น async ตั้งแต่ก่อนเรียก
    Product product = store.get(id);

    // กฎหลักของ Reactive Streams: ห้ามส่ง null เข้า Mono.just() เด็ดขาด
    // (Mono.just(null) จะ throw NullPointerException ทันที ไม่ใช่ตอน subscribe)
    return product != null ? Mono.just(product) : Mono.empty();
}

```

```

public Flux<Product> findAll() {
    // store.values() คืน Collection<Product> ที่มีอยู่แล้วในหน่วยความจำทั้งหมด
    // Flux.fromIterable(...) เป็นแค่ตัว "ห่อ" (adapter) ให้ collection ธรรมดา
    // กลายเป็น reactive stream เพื่อให้ layer อื่นเชื่อม operator ต่อได้
    return Flux.fromIterable(store.values());
}

```

```

public Mono<Product> save(Product product) {
    // store.put(...) คือ side effect ที่เกิดขึ้นทันทีตอน method ถูกเรียก
    // (ต่างจาก operator ปกติของ Reactor ที่มักจะ lazy รอ subscribe ก่อน)
    store.put(product.getId(), product);

    // ห่อผลลัพธ์สุดท้ายด้วย Mono.just(...) เพื่อคืนกลับไปให้ layer บนใช้ต่อ
    return Mono.just(product);
}

```

```

public Mono<Void> deleteById(String id) {
    // ลบออกจาก map จริง ๆ ทันที (ถ้า id ไม่มีอยู่ก็แค่ไม่มีอะไรเกิดขึ้น ไม่ error)
    store.remove(id);

    // Mono<Void> สื่อว่า "งานเสร็จแล้ว แต่ไม่มี payload จะคืน"
    // Mono.empty() ในบริบทนี้ไม่ได้แปลว่า "หาไม่เจอ" เหมือนใน findById()
    return Mono.empty();
}

```

```

public Flux<Product> findByCategory(String category) {
    // เรียกใช้ findAll() เข้า แทนที่จะเขียน logic ดึงข้อมูลใหม่ทั้งหมด
    // เป็นการ "ต่อยอด" (compose) จาก stream ที่มีอยู่แล้ว
    return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));
    /* .filter(...) เป็น operator แบบ lazy จริง ๆ จะยังไม่ execute
    จนกว่าจะมีคน subscribe (ตอน Controller คืนค่ากลับไปให้ Spring)*/
}

```

Service

```
/**
 * ProductService – Business Logic Layer
 *
 * ✅ @Service, Constructor Injection ครบแล้ว (DIP – SOLID)
 * ❌ TODO: เติม method body ให้ครบทุก method
 *
 * หน้าที่: รับ request จาก Controller → เรียก Repository → คืนผล
 * (SRP – แต่ละ class มีหน้าที่เดียว)
 *
 * Hint Operators ที่ควรใช้:
 *   .map(p -> ...)           แปลงค่า
 *   .flatMap(p -> ...)       async transform
 *   .defaultIfEmpty(...)     fallback ถ้าว่าง
 *   .switchIfEmpty(Mono...)  fallback Mono ถ้าว่าง
 */
@Service
public class ProductService {

    // — Constructor Injection (DIP – SOLID) —————
    // Service ไม่ new ProductRepository() เองข้างใน แต่รับผ่าน constructor
    // ทำให้ทดสอบด้วย mock repository ได้ง่าย และ swap implementation ได้ในอนาคต
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public Mono<Product> getById(String id) {
        return repository.findById(id).switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
        /* switchIfEmpty รับ Mono มาแทนที่ ถ้า upstream (findById) จบแบบ empty
        Mono.error(...) หมายถึง "จบด้วยข้อผิดพลาด" แทนที่จะจบเฉย ๆ
        นี่คือนิวที่ business rule ถูกเพิ่ม: "ถ้าไม่เจอ ต้องถือว่าเป็น error"
        ซึ่งเป็นการตัดสินใจของ Service ไม่ใช่หน้าที่ของ Repository */
    }

    public Flux<Product> getAll() {
        // ไม่มี business rule เพิ่มเติม แค่ pass-through ไปยัง repository ตรง ๆ
        // (รักษาโครงสร้าง layer ให้ Controller ไม่ข้าม Service ไปเรียก repository ตรง ๆ)
        return repository.findAll();
    }
}
```

```
public Mono<Product> save(Product product) {  
    // Business rule: ถ้า client ไม่ได้ส่ง id มา (กรณี POST สร้างใหม่)  
    // ให้ระบบ generate id ให้เอง แทนที่จะปล่อยให้ null แล้วพึ่งตอน  
    // เอาไปใช้เป็น key ของ Map ใน Repository  
    if (product.getId() == null) {  
        product.setId(java.util.UUID.randomUUID().toString());  
    }  
    return repository.save(product);  
}
```

```
public Mono<Void> delete(String id) {  
    // ไม่มี validation เพิ่ม (ไม่เช็คว่ามี id นี้จริงไหมก่อนลบ)  
    // ต่างจาก getById ที่ต้องแปลง empty เป็น error  
    return repository.deleteById(id);  
}
```

```
public Flux<Product> getByCategory(String category) {  
    return repository.findByCategory(category);  
}
```

```
public Mono<Double> getDiscountedPrice(String id) {  
    // เรียก getById(id) ของตัวเอง (ไม่เรียก repository ตรง ๆ) เพื่อได้  
    // ผลพลอยได้ฟรี ๆ คือ switchIfEmpty(...) ที่แปลง "หาไม่เจอ" เป็น error  
    // ทำให้ไม่ต้องเขียนเช็คซ้ำอีกรอบใน method นี้  
    return getById(id).map(Product::getDiscountedPrice);  
    // .map(Product::getDiscountedPrice) แปลง Mono<Product> → Mono<Double>  
    // แบบ synchronous (ไม่มี I/O เพิ่ม เพราะเป็นแค่การคำนวณในหน่วยความจำ)  
}
```

Controller

```
/**
 * ProductController – Reactive REST Controller
 *
 * ✅ @RestController, @RequestMapping, Constructor Injection ครบแล้ว
 * ✅ endpoint GET /products/{id} ทำเสร็จแล้วเป็นตัวอย่าง (30%)
 * ❌ TODO: เติม method body ของ endpoint ที่เหลือ (70%)
 *
 * Endpoints ที่ต้องทำทั้งหมด:
 * GET    /products           → Flux<Product>    (ดึงทั้งหมด)
 * GET    /products/{id}      → Mono<Product>  ✅ ตัวอย่างทำแล้ว
 * POST   /products           → Mono<Product>  (บันทึก)
 * DELETE /products/{id}      → Mono<Void>    (ลบ)
 * GET    /products/category/{cat} → Flux<Product> (กรอง)
 * GET    /products/{id}/price → Mono<Double> (ราคาหลังลด)
 */
@RestController
@RequestMapping("/products")
public class ProductController {

    // — Constructor Injection (DIP – SOLID) —
    // Controller ไม่รู้จัก ProductRepository เลย รู้จักแค่ ProductService
    // ทำให้ HTTP layer แยกขาดจาก storage layer อย่างชัดเจน
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }
}
```

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    // Controller ไม่มี logic เอง แค่ "ส่งต่อ" (delegate) ให้ Service ทำงาน
    // Spring WebFlux เป็นคนจัดการ subscribe ให้อัตโนมัติเมื่อคืน Mono/Flux กลับไป
    return service.getById(id);
}
```

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}
```

```
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // @RequestBody บอก Spring ให้ deserialize JSON body ที่ส่งมา
    // เป็น Product object ให้อัตโนมัติ ก่อนส่งเข้า method นี้
    return service.save(product);
}
```

```
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    // คืน Mono<Void> จึง response body จะว่างเปล่าเมื่อทดสอบผ่าน Postman
    return service.delete(id);
}
```

```
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}
```

```
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // ผลลัพธ์ที่ Postman เห็นจะเป็นตัวเลขเดี่ยว (เช่น 35910.0) ไม่ใช่ JSON object
    // เพราะ Mono<Double> ถูก serialize เป็น JSON primitive โดยตรง
    return service.getDiscountedPrice(id);
}
```